Deep Learning and Computation

The software, and a first network in Keras — Lecture 21 (Chapter 10, Part 2)

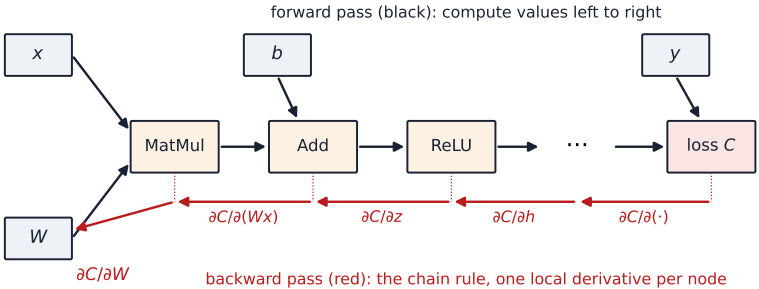
Amir Farbin

Part 2 — Computing it: the software

Where we are

- Lecture 18: what deep networks do. Lecture 19: classical ML in scikit-learn, on SUSY
- Today: **the software** that builds and trains deep networks, and our first network in **Keras**
- Same data, same question: is this collision signal or background?
- The notebook is a template: data, model, compile, fit, check, evaluate, ROC. Lab 9 starts from it

Numbers versus symbols

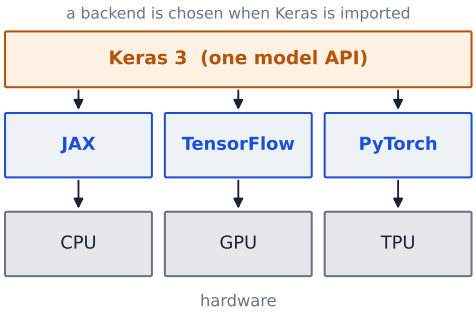


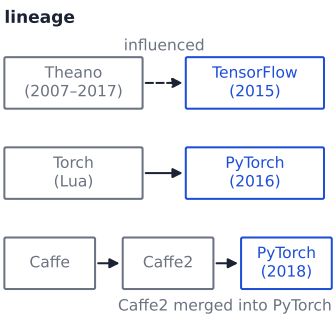
- **NumPy** computes `D = A @ B + C` the moment you type it
- A **framework** first builds the **computation graph** from **symbols** (tensors with no numbers yet), optimizes it, runs it on a **GPU**, and **differentiates** it

Why symbols: the gradient

- Training needs $\partial L/\partial w$ for **every** weight w: the **gradient** of the **loss** L
- The loss goes through $f(Wx + b)$ again and again, layer after layer: written out, it is enormous
- And it changes **every time you change the architecture**
- **Automatic differentiation:** the framework derives the gradient from the graph, exactly, every time

The stack in 2026



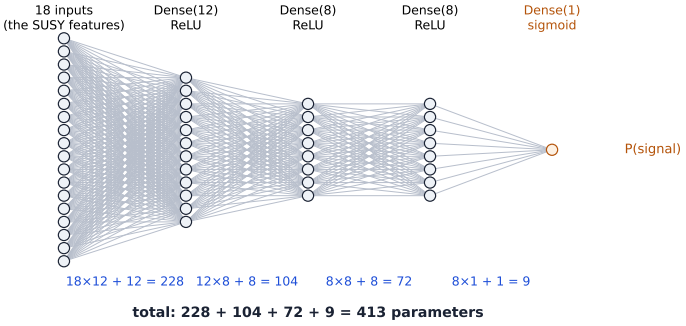


- **Engines (backends):** PyTorch, TensorFlow, JAX: compute and differentiate tensors on CPUs and GPUs
- **Keras 3** describes the model and the loss, and runs on any of them

What Keras is

- You **describe the architecture** in Keras: layers, connected
- Keras **writes the mathematical expression** for the loss, and the engine computes it
- "When you're describing a model, what you're really doing is describing how to construct the mathematical expression for a loss function"
- A **layer** is a function: tensor in, tensor out. Layers are **Lego pieces**: stack them
- Every architecture, from a dense network to a transformer, is implemented as layers

Our first network



- Design rule: **18 features in, 1 number out**; free in between
- Hidden layers: **ReLU**, $\max(0, x)$. Output: **sigmoid**, a number in $(0, 1)$ read as P(signal)
- Deliberately tiny, so it trains in seconds. Real models: hundreds of units per layer

Sequential: a stack of layers

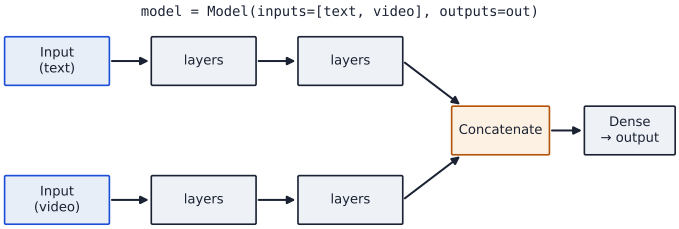
```
model = Sequential()
model.add(Dense(12, input_dim=18, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(8, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
```

- **Dense layer:** every input connected to every **unit** (neuron); computes $f(Wx + b)$
- W: the **weights**, b: the **biases**, f: the **activation function**
- `Sequential` pushes the data through the layers in order. Easy to read, but only a straight line

Counting parameters

- A dense layer has $n_{in} \times n_{out}$ weights and n_{out} biases
- $18 \times 12 + 12 = 228$, $12 \times 8 + 8 = 104$, $8 \times 8 + 8 = 72$, $8 \times 1 + 1 = 9$: total **413**
- `model.summary()` lists each layer's **output shape** and **parameter** count. Check them by hand
- **Output shape** `(None, 12)`: `None` is the **batch**, any number of examples pushed through at once
- 413 parameters $\times$ 4 bytes $=$ 1.61 KB. Today's large models: billions of parameters, gigabytes

The functional API: draw a graph




```
in_x = Input(shape=(18,))
x = Dense(12, activation='relu')(in_x)     # call the layer on a tensor
out_x = Dense(1, activation='sigmoid')(x)
model = Model(in_x, out_x)                 # pick the input and the output
```

Read the summary

- The notebook's third model writes the activation as its own step: `x = relu(Dense(8)(x))`
- Same computation as `Dense(8, activation='relu')`: without an activation, a layer is linear
- Its summary must say **413** again. Python keeps the last `model`, and **that** one is trained
- Until this term it said **341**: one hidden `Dense(8)` had gone missing (fixed now)
- Lesson: compare `summary()` tables. They catch a missing layer at a glance

A name or an object

- `optimizer='adam'`: a **string** asks Keras to build the object with its defaults (a **factory**)
- To change a setting, build the object yourself and pass it in:

```
model.compile(loss='binary_crossentropy',
              optimizer=keras.optimizers.Adam(learning_rate=1e-3),
              metrics=['accuracy'])
```

- The same holds for losses, metrics, activations, initializers

compile: the loss is the point

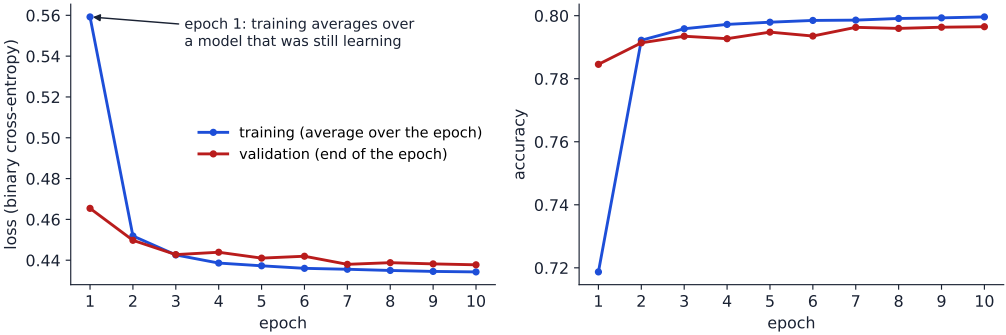
- `compile` builds the expression training will minimize: the **loss**, a single number
- **Binary cross-entropy:** the loss for two classes with a sigmoid output; large when confidently wrong
- **Optimizer:** turns gradients into weight changes. **Adam** adapts the step for each parameter
- **Metric:** watched, not optimized. **Accuracy** = fraction right with the output cut at 0.5

fit: two loops

```
for each epoch:              # one pass over the data
    for each batch:          # 2,048 examples
        forward: outputs and loss
        gradient of the loss for every parameter
        optimizer step: change the parameters
    validation: loss and metrics, no updates
```

- **Epoch:** one pass over all the training data. **Batch:** the examples behind one update (a **step**)
- 500,000 / 2,048 → **245 steps** per epoch. Batch sizes are powers of two to suit the hardware
- **Stochastic gradient descent:** each step's gradient comes from a random batch, not the whole set

Learning curves



- Epoch 1: training is an **average over the epoch**, while the model was still learning; validation is computed **at the end**
- By epoch 10 the validation loss has nearly flattened: close to a **plateau**

Over-training, and what to do about it

- Train long enough with enough **capacity** (parameters): training loss keeps falling, validation turns up
- That is **over-training** (over-fitting): the model memorizes the training sample
- "I purposely over-train, but I save my model along the way and then pick a model before it over-trained"
- **Callbacks** run during training: `ModelCheckpoint` saves the model, `EarlyStopping` stops when validation stops improving
- The validation curve wiggles more: it is computed on fewer events

Validation is not test

- The notebook validates on `X_Test`, then `evaluate(X_Test)` returns **exactly** epoch 10's validation numbers
- **Validation sample:** used while you work: to watch training, stop, and compare models
- **Test sample:** used **once**, at the end, on a model no choice was based on
- Comparing models in Lab 9 is a choice: keep a third slice for the test

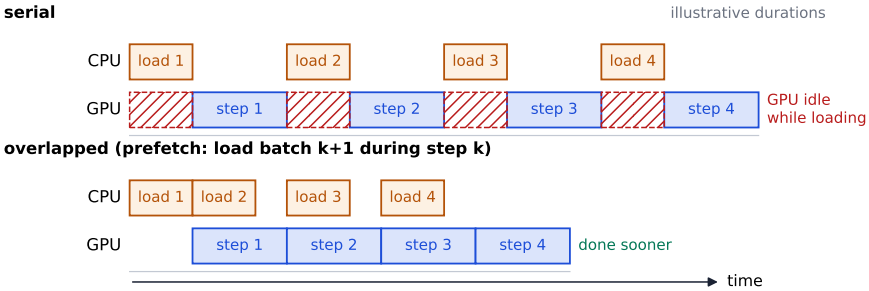
Using the model

- `model(X)`: all rows at once; returns a tensor. `model.predict(X)`: in batches; returns NumPy
- The sigmoid output: a number in $(0, 1)$, loosely the probability of signal. Choose the threshold yourself
- Plot the output for signal and background separately: 18 features became **one**
- **Baseline:** LDA on the same features gave AUC ≈ 0.84; this tiny network gives **0.87**
- Always beat something simple first. If a deep model can't, that points to a technical problem

keras.io: where to look

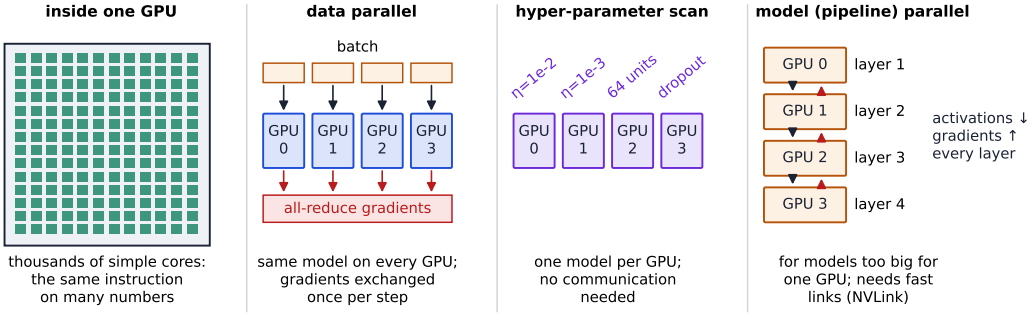
- **Developer guides:** the functional API, training loops, callbacks, transfer learning
- **API reference:** layers, activations, initializers, regularizers, preprocessing layers, optimizers, losses, metrics, data loading
- **Code examples:** "almost anything that you can think of", many reproducing a paper: copy, then modify
- **Pre-trained models** (Keras Applications): don't train images from scratch

Keeping the GPU fed



- Data bigger than memory, in many files: read, decode, shuffle, augment, normalize
- The CPU prepares batch $k+1$ while the GPU runs batch k (`tf.data`, PyTorch `DataLoader`)

Distributed training



- **Data parallel:** split the batch, average the gradients each step
- **Model parallel:** split the model when it doesn't fit; activations cross between devices every layer

From choices to code

- Layer sizes and activations: `Dense(units, activation="relu")`
- Output and cost: `Dense(1, activation="sigmoid")`, `loss="binary_crossentropy"`
- Optimizer: `optimizer=keras.optimizers.Adam(learning_rate=1e-3)`
- `batch_size`, `epochs`, `validation_data`, `Dropout(rate)`, callbacks

Vocabulary

- **framework, backend (engine):** TensorFlow, PyTorch, JAX. **Keras:** writes the model's math for them
- **tensor:** an array with any number of dimensions. **computation graph, symbol, automatic differentiation, GPU**
- **layer, Dense (fully connected) layer, unit (neuron), weights, biases, parameters, trainable**
- **activation function, ReLU, sigmoid. Sequential API, functional API, Input, Model**
- **output shape, batch dimension (`None`), `summary()`. factory:** a name that builds an object
- **compile, loss, binary cross-entropy, optimizer, Adam, metric, accuracy**
- **fit, epoch, batch, batch size, step, stochastic gradient descent, history, learning curve, plateau**
- **capacity, over-training (over-fitting), callback, early stopping, model checkpoint**
- **validation vs test sample, evaluate, predict, threshold, baseline, ROC, AUC**

What comes next

- Lecture 22: the same template on a table from your Kaggle project, with **normalization** inside the model
- Lab 9: more units, more layers; raw vs derived features; add them to Lab 8's comparison
- Then: the pieces (activations, costs, optimizers, regularization) and images (Lecture 23)